

Multi-Account UI/UX — All Client Surfaces (to-be design §23)

Revision: 1 **Last modified:** 2026-07-10T11:18:54Z **Status:** to-be design (planning only — NOT implemented); builds on the as-is audits 10/11/12; implementation gated on operator approval (superpowers brainstorming hard-gate per 00_INDEX §2) **Authority:** operator mandate 2026-07-10 (multi-account BACKGROUND research); realises 00_INDEX §3 doc 23_* **Scope:** the two in-scope UI surfaces — the operator **dashboard** (dashboard/) and the **OTA Manager** (clients/ota-manager/, which doubles as the server-hosted /manager web console). Sign-in split, post-login account selection, and the super-admin console, all per surface.

0. Grounding + guardrails (read first)

This doc is a **design proposal**, not code. Every current-state claim is cited to a file:line established by the as-is audits (12_existing_ui_surfaces_and_design_system.md is the authoritative current-state for this doc — it is **cited, never contradicted**), or re-verified directly this session. Recommendations carry tradeoffs; nothing is silently decided (§11.4.6, §11.4.66).

Entity-shape SSOT. The authoritative account / user / membership entity shapes are the job of 20_target_multitenancy_data_model.md (planned per 00_INDEX §3; **not yet authored** at this doc's revision). This doc therefore treats those entities as **opaque** and, wherever a concrete field name is needed for UI wiring, uses **only shapes already present in the codebase** — never an invented, conflicting account/membership schema:

- Role = 'viewer' | 'operator' | 'admin' | 'device' | 'super_admin' (clients/ota-manager/src/lib/api-client.ts:323).
- ProjectAccess { project_id: string; role: Role } (api-client.ts:325-328).
- Project { project_id, name, description?, created_at, updated_at } (api-client.ts:568-574).

- Auth-store
`user { id, email, display_name, avatar_url, roles[], permissions[] } (clients/ota-manager/src/stores/auth-store.ts).`
- The established `account_id` snake_case convention (00_INDEX §4: `account` → `projects` → OTA updates).

Where this doc shows an `Account {...}` or `Membership {...}` field list it is marked **illustrative (SSOT = 20_*)** — it is a UI-wiring sketch, not an authoritative schema. When `20_*` lands, the field names here are reconciled to it (not the reverse).

Current-state facts this design attaches to (re-verified this session):

Fact	Evidence
Both surfaces already ship a working sign-in + session + RBAC	12_* §4
OTA Manager already defines <code>super_admin</code> + <code>ProjectAccess</code> + <code>isSuperAdmin()</code>	<code>permissions.ts:12-18,143</code> ; <code>api-client.ts:323-328</code>
OTA Manager has a (MOCK) project switcher — the exact account-switcher seam	<code>project-switcher.tsx:13-16</code> (<code>MOCK_PROJECTS</code> , local <code>useState</code> only)
<code>use-projects.ts</code> already anticipates a <code>GET /projects</code> server endpoint with placeholder	<code>use-projects.ts:21-45</code>
Dashboard Role union <code>OMITS super_admin</code> — a real divergence	<code>dashboard/src/types/api.ts:12</code> (<code>"admin"</code> \ <code>"operator"</code> \ <code>"viewer"</code> \ <code>"device"</code>)
Neither surface has any <code>account/tenant</code> element	12_* §4 (0 refs in either <code>src</code>)
Host-render proof harnesses ALREADY EXIST for both surfaces	<code>clients/ota-manager/visual/</code> + <code>dashboard/hostrender/</code> (re-verified §4 below)

1. Sign-in flows — super-user vs regular user

The mandate (00_INDEX §1 item 5) wants “super-user sign-in, user sign-in, and account selection after sign-in” — i.e. a **split** between the System owner (super-admin) and a normal account user. There are two ways to realise the split; they differ in *where* the split happens.

1.1 What differs functionally (the destination, regardless of mechanism)

	Super-user (super-admin)	Regular user
Identity role	carries super_admin (permissions.ts:143 isSuperAdmin)	one or more of viewer/operator/admin scoped per account
Landing after auth	global System console (all accounts, all users, all permissions)	account selection → then the account’s project/OTA workspace
Token account scope	System scope — cross-account (no single account_id, or a system flag)	scoped to the active account (account_id claim)
Account creation	only super-admin may create accounts/users (00_INDEX §1 item 2 — no self-registration wizard at this stage)	none

1.2 Two mechanisms — recommendation with tradeoffs

Option A (RECOMMENDED) — one sign-in form, POST-auth routing split. A single login form per surface (the ones that already exist: dashboard/src/screens/LoginScreen.tsx and clients/ota-manager/src/features/auth/login-page.tsx). After successful auth the app reads the role claim and **routes**: super_admin → global console; everyone else → account selection (§2). This matches the industry pattern — one identity-scoped session, one login, the “super vs user” distinction

resolved from the token's role, not from a second credential endpoint (WorkOS, Clerk — §Sources). Reuses the existing login UI verbatim; the split is a guard/router concern, not a new form.

- **Pros:** minimal new UI (no second login screen to build or host-render-prove); one session/refresh code path; smallest attack surface; matches the WorkOS/Clerk “active-org state, one login” model the rest of this design assumes.
- **Cons:** less literal to the mandate's “super-user sign-in vs user sign-in” phrasing — the two sign-ins are the **same form** with a post-auth divergence, not two doors.

Option B — a distinct super-admin sign-in entry. A separate super-admin login route (e.g. a /admin entry or a “System sign-in” toggle), optionally on a separate host/deployment. Literal to the mandate; enables defense-in-depth (super-admin auth can demand stronger MFA, IP allow-listing, a separate rate-limit posture).

- **Pros:** physically separate super-admin door; pairs naturally with a future separately-deployed super-admin console; clearer audit story.
- **Cons:** a second login form to build + host-render-prove on the super-admin surface; duplicated session/refresh logic; two sets of §11.4.170 golden fixtures.

Recommendation: ship **Option A** now (post-auth routing split), and keep **Option B** open as a hardening step if the operator later wants a physically separate, stronger-MFA super-admin console. This is an **operator decision (§11.4.66)** — flagged for 21_authz_rbac_superuseradmin.md to bind alongside the token/permission model. This doc does not silently pick B.

1.3 Session / token extension for the account claim (UI-side view; authoritative model = 21_/22_)

The session model both surfaces already run is an **identity-scoped session** (in-memory access token + rotating refresh token — dashboard/src/auth/AuthContext.tsx; clients/ota-manager/src/stores/auth-store.ts). The multi-account extension follows the recommended industry pattern (WorkOS — §Sources): keep the refresh token

identity-scoped, and on account selection/switch **exchange it for a short-lived access token whose claims include the chosen account_id**. UI-side that means:

- Auth store gains an **activeAccountId** (+ the user's accounts[] membership list) — a new piece of state distinct from the identity. `auth-store.ts` already persists `roles[]/permissions[]`; `activeAccountId` sits beside them.
- Switching account (§2) triggers a token re-exchange, then invalidates account-scoped caches (§2.3). Super-admin's token carries the System scope instead of a single `account_id`.
- The **account claim is derived on demand**, never embedded in the long-lived credential — so a compromised/rotated session cannot silently act in the wrong account.

The exact claim names, the token exchange endpoint, and the System-scope encoding are **21_*/22_*'s** to define; this doc only states the UI-visible state (`activeAccountId`) and the switch → re-exchange → cache-invalidate sequence.

2. Account selection AFTER sign-in

2.1 The post-login picker (the “choose active account” step)

After a **regular** user authenticates, resolve the active account:

- **0 memberships** → dead-end error state (“no account access — contact your System administrator”); never a silent blank. (Creation is super-admin-only — 00_INDEX §1 item 2.)
- **exactly 1 membership** → **auto-select and skip the picker** (Clerk pattern — §Sources); the user lands directly in that account's workspace.
- **2+ memberships** → show the **account picker** screen: a list of the user's accounts (name + the user's role in each — Slack pattern surfaces the per-org role), select one to set `activeAccountId`.
- **returning user** → restore the last-active account from persisted `activeAccountId` (skip the picker), with a **“switch account” control always available** (§2.2). A super_admin **bypasses** to the global console, with an explicit “act within account X” affordance to drop into a specific account context.

2.2 Where the switcher attaches — AccountSwitcher above ProjectSwitcher (two-level context)

The context model is **account** → **project** (00_INDEX §4). The switcher is a second dropdown that mirrors the existing project switcher **exactly**, placed **above** it, so the user reads top-down: *which account* → *which project*.

- **OTA Manager (has the precedent):** `features/layout/project-switcher.tsx` is a shadcn DropdownMenu (FolderKanban trigger, checkmarked items) rendered in the sidebar header when expanded (`sidebar.tsx:57` renders `<ProjectSwitcher />`; the collapsed state at `sidebar.tsx:39-55` shows the logo, not switchers). The new **AccountSwitcher** is a near-copy of `project-switcher.tsx` (a Building2/Users trigger instead of FolderKanban), placed **above** `<ProjectSwitcher />` in that same sidebar header block, or in `topbar.tsx` (currently only theme toggle + user menu — `topbar.tsx:32-68`). It is backed by a new **hooks/use-accounts.ts** that mirrors `hooks/use-projects.ts:21-45` (a GET `/accounts` query with a placeholder until the server implements it).
- **Dashboard (greenfield — no switcher of any kind):** `AppShell.tsx:36-44` is a fixed route list with no project/account concept. Port the OTA-Manager pattern: an AccountSwitcher in the `AppShell.tsx` header user area (`AppShell.tsx:72-87`), plus the post-login picker route.

2.3 Account-switch behaviour (data re-scoping)

When the active account changes: (1) re-exchange the token for the new `account_id` claim (§1.3); (2) **the project switcher re-scopes** — its list is now the projects of the newly active account, and the current project selection resets; (3) **invalidate account-scoped caches** — TanStack Query keys for account-scoped resources must include `activeAccountId`, so switching accounts refetches rather than showing stale cross-account data (the Slack “clear cache on switch” guidance — §Sources; also a tenant-isolation guardrail deferred to `26_security_threat_model.md`). `use-projects.ts`’s `projectKeys` (`use-projects.ts:11-17`) gains `activeAccountId` in its key.

2.4 Recommendation with tradeoffs

Recommend **auto-select-single + persist-last + always-available switcher** (§2.1). The one tradeoff to flag: persisting the last-active account is a small convenience-vs-explicitness call — a returning user

lands in the last account rather than re-choosing. Mitigation: the switcher is always visible and the active account is always labelled, so the current context is never ambiguous (a §11.4.162 no-hidden-state concern). If the operator prefers an explicit re-choose every session, drop the persistence — flagged for 21_*

3. Super-admin console (administer all accounts + users + permissions)

The super-admin console is the System-owner surface: create/administer **all accounts**, **all users**, memberships, and **per-account role/permission assignment** — “ultimate flexibility” (00_INDEX §1 item 6). It is gated by `isSuperAdmin()` (`permissions.ts:143`), which already exists.

3.1 Screens

Screen	Purpose	Reuses / generalises
Accounts list	all accounts; search/sort; row → detail	new; list pattern like the OTA-Manager tables
Account create / edit	super-admin-only creation (no self- registration — 00_INDEX §1 item 2); edit name/ status	new dialog/form (react- hook-form + zod already in OTA Manager)
Users list	all users across the System; search/sort	new; no persisted User entity exists yet (10_) — <i>depends on 20_ users table</i>
User create / edit	provision a user (super-admin- only)	new
Membership assignment	assign a user to account(s) with a role (Membership, illustrative — SSOT 20_*)	generalises ProjectAccess{project_id, role} → AccountAccess{account_id, role}
	who-can-do-what within an account	reuses the RESOURCE_PERMISSIONS shape

Screen	Purpose	Reuses / generalises
Per-account role / permission assignment	— the permissions matrix (§3.2)	(permissions.ts:30-42) as the display model

3.2 The permissions matrix UI (“ultimate flexibility to the maximum”)

The permission model itself (RBAC vs role+scope vs full ABAC) is **OQ-2**, resolved in 21_authz_rbac_superadmin.md — this doc designs the **UI surface** for it, not the model.

The natural, already-present shape is a **roles × (resource × action) grid**: the OTA Manager already ships exactly this data as RESOURCE_PERMISSIONS (permissions.ts:30-42 — rows = resources devices/artifacts/releases/..., columns = actions view/create/update/delete, cells = minimum role). The matrix UI renders that grid **per account**, letting the super-admin see and adjust who-can-do-what.

- **Recommendation**: ship the matrix over a **role catalog + per-account role assignment** (RBAC) as the pragmatic default — it maps 1:1 to the existing Role hierarchy (permissions.ts:12-18) and the existing matrix data, and is the industry default for multi-tenant B2B (§Sources: roles shared at the app level, *assignment* tenant-specific).
- **Tradeoff / flag**: true per-permission ABAC (arbitrary resource:action grants per user) is more flexible but a heavier model + heavier UI. The matrix UI can present RBAC now and grow an “advanced (custom permissions)” mode later. The model choice is ****21_***s (OQ-2)** — this doc does not pick it; it designs a matrix that can render either.
- **No-overlap constraint (§11.4.162)**: a resource×action grid is wide — it **MUST** live in a horizontal-scroll container inside the constrained content area and must not collapse/clip header labels; proven by the OCR/vision layout oracle (§4) reading every cell + header bound.

3.3 Where it attaches

- **OTA Manager**: a new TanStack route (e.g. /admin/*) + a sidebar.tsx nav item gated by isSuperAdmin (the navItems array at sidebar.tsx:17-24 is the seam); a new feature folder src/features/admin/ (accounts / users / permissions). Because the server serves

this build at /manager (server/internal/api/embed.go:56), the console reaches the web console automatically on re-embed.

- **Dashboard:** new route(s) + entries in the fixed AppShell.tsx NAV (AppShell.tsx:36-44) gated by a RoleGate allow={"super_admin"} (AppShell.tsx:29) — **which requires adding super_admin to the dashboard Role union first (§5).**
-

4. MANDATORY constraints — OpenDesign tokens (§11.4.162) + host-rendered visual proof (§11.4.170)

4.1 OpenDesign design tokens (§11.4.162) — light + dark, no hardcoded hex

Every new/changed element MUST consume the helix-ota OpenDesign token package (design-systems/helix-ota/tokens.css, light+dark first-class) — both surfaces already vendor it (OTA Manager src/styles/opensdesign-tokens.css; dashboard src/styles/tokens.css — 12_* §2.2). Concretely for the tenancy UI:

- **No hardcoded hex** in any new AccountSwitcher / picker / console / matrix element — all color/spacing/typography via var(--token). shadcn components in OTA Manager already render over the token layer (0 hardcoded hex in src — 12_* §2.2), so new shadcn-based screens are token-clean by construction.
- **Dashboard has a 58-hardcoded-hex debt** (12_* §2.2). Recommendation: **finish repointing those to var(--token) before layering tenancy UI**, so new screens start token-clean. The **fixed dark-navy brand-chrome header** hexes (AppShell.tsx:96-107 block comment; :113/:114/:121/:127/:129) are a **deliberate, documented exception** — a self-contained dark surface fixed in both themes — and are **excluded** from the vendoring task (tokenizing them would break the bar in light mode). New tenancy screens live in the themed body, not the fixed chrome, so they use tokens regardless.
- **Light + dark both required** for every new screen and every state.

4.2 No overlap / no label-over-label (§11.4.162)

Specific risk surfaces for this design:

- The **AccountSwitcher above the ProjectSwitcher** in the constrained sidebar (`sidebar.tsx:34: w-60 expanded / w-16 collapsed`) must not collide with each other or the logo (collapsed state renders the logo, not the switchers — `sidebar.tsx:39-55`), and long account/project names must truncate (the project switcher already truncates — `project-switcher.tsx:25-27`), not overflow.
- The **permissions matrix** must not overflow horizontally (§3.2 scroll container) and its header row must not overlap the first data row.
- The **account picker** list rows must not clip role labels.

All proven by the OCR/vision layout oracle (§4.3), which reads rendered text + control bounds (no overlap / label-over-label / clipping / off-screen).

4.3 Device-independent host-rendered pixel proof (§11.4.170) — dual-validated

Every screen × state × {light, dark} MUST be proven by **device-independent host-rendered pixels** (the real component rendered to a PNG on the host — no device/emulator), **dual-validated** by (i) a **golden image-diff** AND (ii) an **OCR/vision oracle** reading rendered text + labels + control bounds. **Value/token-equality unit tests are FORBIDDEN as the sole proof** (§11.4.170 forensic: hex/sp/dp value-equality tests stayed GREEN while the operator opened a visibly-broken screen); they MAY supplement, never substitute the rendered-pixel proof.

The harnesses already exist — the new screens add fixtures to them (re-verified this session):

- **OTA Manager** — `clients/ota-manager/visual/`: `harness.tsx + run-all.mjs` render components to PNG on the host (Storybook-class); **oracle-diff.mjs = golden image-diff**; **oracle-ocr.mjs = the OCR/vision layout oracle**; `vite.harness.config.ts`. It also ships a **dashboard variant** (`harness-dashboard.tsx`, `run-all-dashboard.mjs`) — so the OCR/vision oracle can cover dashboard screens too.
- **Dashboard** — `dashboard/hostrender/`: Playwright specs (`login.hostrender.spec.ts`, `screens.hostrender.spec.ts`) with

- snapshots/ dirs (Playwright toHaveScreenshot = golden image-diff)
+ playwright.hostrender.config.ts.

Test approach (Playwright / Storybook-class host-render for these React SPAs): render each new tenancy component to a host PNG via the existing harness, per state, in **both themes**; run **both** oracle layers (golden-diff + OCR/vision). New screens to prove, per state × {light, dark}:

Screen	States to prove
Regular sign-in (unchanged form, re-proven)	idle · validation-error · submitting
Account picker	multi-account list · no-membership error · (single-account = auto, no screen)
AccountSwitcher (dropdown)	closed · open · active-account checked · long-name truncation
Super-admin: accounts list	populated · empty · loading
Super-admin: account create/edit dialog	empty · filled · error
Super-admin: users list + user create/edit	populated · membership-assignment open
Super-admin: permissions matrix	full grid · horizontal-scroll (no clip/overlap)
(Option B only) super-admin sign-in	idle · error

Honest boundary (§11.4.170): these are **host-rendered** pixels (device-independent) — the proof is that the real component renders correctly, dual-validated; it is **not** a running end-to-end wire proof (that is §11.4.169/27_*).

5. Role-vocabulary reconciliation (dashboard $\rightleftharpoons$ OTA Manager) — flag for 21_*

Primary divergence (12_* §6, re-verified): the OTA Manager defines Role = 'viewer' | 'operator' | 'admin' | 'device' | 'super_admin' (api-client.ts:323) with a full ROLE_HIERARCHY including super_admin:3 (permissions.ts:12-18), but the **dashboard's Role union OMITs super_admin** — dashboard/src/types/api.ts:12 is "admin" | "operator"

| "viewer" | "device". The super-admin console **cannot be gated on the dashboard** until `super_admin` is added to that union (and to the `TokenResponse.roles` type, `dashboard/src/types/api.ts:24`).

Recommendation: add `super_admin` to the dashboard `Role` union and align both surfaces to a **single shared role vocabulary** before adding any super-admin UI. This is a small, low-risk type change but load-bearing for §3.3's dashboard gating.

Secondary inconsistency found this session (flag, don't fix here): the OTA Manager `sidebar.tsx` nav gates reference a role literal `"developer"` (`sidebar.tsx:20-21`, `roles: ["admin", "developer"]`) that is **not** a member of the `Role` union at all (`api-client.ts:323`). `RoleGate` is UX-only (the server enforces authoritatively — `AppShell.tsx:2-3` notes the same for the dashboard), so this is a latent nav-gate typo, not an auth hole — but it means those nav items never match by that literal. Reconciling the role vocabulary should sweep this too.

Deferred to 21_authz_rbac_superadmin.md: the authoritative single role vocabulary, the `super_admin` System-scope semantics, and whether `AccountAccess/membership` rows replace the per-project `ProjectAccess` shape. This doc only flags the UI-blocking divergence.

6. Per-surface extension-point table (concrete file/component seams)

Where sign-in + account-selection + super-admin console attach, per surface.

6.1 OTA Manager (`clients/ota-manager/`) — the richer, further-along surface

Concern	Concrete seam (file:line)	Change
Sign-in (regular; super via post-auth split, §1.2 Option A)	<code>src/features/auth/login-page.tsx</code> (+ <code>login-form.tsx</code> , <code>auth-guard.tsx</code>)	keep single form; add post-auth role routing in the guard/router
Session + active-account state (§1.3)	<code>src/stores/auth-store.ts</code>	add <code>activeAccountId</code> + <code>accounts[]</code>

Concern	Concrete seam (file:line)	Change
Thread account claim into requests (§2.3)	src/lib/api-client.ts	beside roles[]/ permissions[] attach active account_id; token re- exchange on switch
Post-login account picker (§2.1)	new TanStack route + src/ features/auth/auth-guard.tsx	new picker screen; auto- select single, restore last
AccountSwitcher (§2.2)	new src/features/layout/ account-switcher.tsx mirroring project-switcher.tsx; mount above <ProjectSwitcher/> in sidebar.tsx:57 (or topbar.tsx:32)	new component
Accounts data hook (§2.2)	new src/hooks/use-accounts.ts mirroring hooks/use- projects.ts:21-45	new GET / accounts query + placeholder
Account-scoped cache keys (§2.3)	src/hooks/use- projects.ts:11-17 (projectKeys)	include activeAccountId in keys
Super-admin console (§3.3)	new route /admin/* + sidebar.tsx:17-24 nav item gated by isSuperAdmin (permissions.ts:143); new src/ features/admin/	new screens
Permissions matrix (§3.2)	new admin screen reusing RESOURCE_PERMISSIONS (permissions.ts:30-42) as display model	new component
Membership model (§3.1)	generalise ProjectAccess{project_id,role} (api-client.ts:325) → AccountAccess{account_id,role} (illustrative — SSOT 20_*)	type + UI

6.2 Dashboard (dashboard/) — greenfield for tenancy; needs alignment first

Concern	Concrete seam (file:line)	Change
Role divergence (§5) — do first	dashboard/src/types/api.ts:12,24	add super_admin to Role union + TokenResponse.roles
Sign-in (regular; super via post-auth split)	dashboard/src/screens/LoginScreen.tsx (OAuth2 ROPC)	keep single form; post-auth role routing
Session + active-account state (§1.3)	dashboard/src/auth/AuthContext.tsx (hasRole() already present)	add active-account state
Post-login account picker (§2.1)	new route + ProtectedRoute (AppShell.tsx:19-26)	new picker screen (no precedent — port OTA-Manager pattern)
AccountSwitcher (§2.2)	new component in the AppShell.tsx:72-87 header user area	port pattern (no switcher precedent here)
Super-admin console (§3.3)	new route(s) + AppShell.tsx:36-44 NAV entries gated by RoleGate allow={"super_admin"} (AppShell.tsx:29)	new screens (needs §5 first)
Token-vendoring debt (§4.1) — do before new screens	58 hardcoded hex (12_* §2.2), excluding fixed chrome AppShell.tsx:96-107,113,114,121,127,129	repoint to var(--token)

6.3 Server /manager (C) and Android/bridge (D/E)

- **Server /manager (C):** no separate UI work — it is the OTA-Manager build; changes reach it automatically on re-embed (server/internal/api/embed.go:56 //go:embed manager-dist/*).
- **Android agent + bridge (D/E): no UI to extend** — both are headless Kotlin (12_* §1 Surfaces D/E: 0 Compose/MaterialTheme markers). The device-side “setup wizard” (00_INDEX §1 item 10) would be a **new Compose surface** needing a Kotlin token bridge off helix-ota/tokens.css + Roborazzi/Paparazzi host-render proof

(§11.4.170) — that is `25_device_side_update_client.md`'s scope, not an account-UI target here.

Sources verified 2026-07-10

External best-practice research per §11.4.8 / §11.4.99 (multi-tenant SaaS sign-in + workspace/account-switcher UX + super-admin/permission patterns). Findings cross-referenced, not copied; each claim above that leans on an external pattern points here.

- **WorkOS — “Multi-tenant session management: isolation patterns that actually work.”** <https://workos.com/blog/multi-tenant-session-management> — verified the recommended pattern: ONE identity-scoped session with in-session org switching; the “active organization” is separate backend state, NOT embedded in the long-lived credential; the identity-scoped refresh token is **exchanged on demand for a short-lived access token whose claims include the chosen org_id**. Grounds §1.3 (account claim derived on demand) + §2.3.
- **Clerk — “What is multi-tenancy and why it matters for B2B SaaS.”** <https://clerk.com/blog/what-is-multi-tenancy-and-why-it-matters-for-B2B-SaaS> — verified: users may belong to multiple organizations under a single identity; every B2B org has an admin/owner managing users/memberships/permissions. Grounds §2.1 (multi-membership) + §3. Negative finding (§11.4.99): this page does **not** give concrete org-switcher UI placement or post-login auto-select guidance — those rest on the WorkOS + LoginRadius sources and the in-repo precedent, not this page.
- **LoginRadius — “Access Control SaaS Guide for B2B & Multi-Tenant Platforms.”** <https://www.loginradius.com/blog/engineering/rbac-saas-multi-tenant-b2b-platforms> — verified: RBAC must be tenant-aware (a role never exists without organizational context); roles/permissions are shared at the app level while **assignment is tenant-specific** (same user, different roles per tenant); permissions as granular action strings composed into roles. Grounds §3.2 (RBAC-first matrix; assignment per account).
- **Sequency — “How to Build an Admin Panel for a SaaS Product.”** <https://www.sequency.com/blog/how-to-build-saas-admin-panel> — verified the super-admin/back-office console pattern: a centralized interface for operators to manage customers, permissions, and settings. Grounds §3.1 (screen set).

General UX corroboration (org switcher kept visible in the nav; the user's per-org role shown; caches cleared on switch — Slack-style) surfaced consistently across the WorkOS/Clerk/LoginRadius search corpus and is applied in §2.2/§2.3.

Honest boundary (§11.4.6)

- **Design proposal, NOT implemented.** No component here exists yet; every “new” seam is a plan. No code was written, no pixels rendered, no git run under this doc.
- ****Entity shapes are 20_*’s, not this doc’s.**
20_target_multitenancy_data_model.md is the SSOT for account/user/membership and is not yet authored at this revision. Every Account{...}/Membership{...} field list here is marked illustrative** and reconciles to 20_* when it lands — this doc never asserts an authoritative account/membership schema, and the only field shapes stated as fact are ones already present in the codebase (§0).
- ****The permission MODEL is 21_*’s (OQ-2), not this doc’s. §3 designs the console + matrix UI surface****; whether the underlying model is RBAC / role+scope / ABAC is deferred. The recommendation (RBAC-first matrix) is a recommendation with a stated tradeoff, not a decision.
- **The sign-in split mechanism (Option A vs B) is an operator decision (§11.4.66).** This doc recommends A with a documented rationale and keeps B open; it does not silently pick.
- **Host-render harness existence is verified; per-screen PASS is not.** §4 confirms the golden-diff + OCR/vision oracle harnesses exist (file-level), and specifies the fixture matrix; it does **not** claim any tenancy screen renders correctly — that is the §11.4.170 proof to be produced during implementation, out of scope for a planning doc.
- ****UNCONFIRMED (carried from 12_*):**** “OpenCode = coding agent, OpenDesign = token system” (OQ-1) rests on repo evidence, flagged for operator confirmation (§11.4.66); it does not affect this doc’s token guidance, which targets the already-vendored `helix-ota/tokens.css`.